

NIDS

Network State Detection and Forecasting

Product Description

1 Overview

NIDS is a fully offline network intrusion detection and short-horizon forecasting system. It moves past per-flow benign/malicious labelling towards *predictive* defence: it learns how the aggregate state of a network evolves over time and forecasts the next several state windows, then attaches a restrained MITRE ATT&CK interpretation and a per-prediction feature attribution so a defender can see *why* a call was made.

Everything runs on a local machine. There is no cloud dependency, no telemetry, and no external API call at inference time.

2 The problem it addresses

Traditional classifiers treat each traffic flow in isolation and map it to a single label. That discards the temporal structure of an intrusion — the order in which ports are probed, the way SYN bursts precede floods, the timing of reconnaissance before lateral movement. An intrusion is a process that unfolds over time. NIDS models that process: it aggregates traffic into fixed time windows, learns the state-transition behaviour $P(S_{t+1} | S_t)$, and rolls it forward.

3 Pipeline

1. **Capture / ingest.** Live capture via `dumcap` (Windows) or `tcpdump` (Linux, macOS), or offline upload of a `.pcap/.pcapng/.csv` file.
2. **Flow features.** The Scapy-based `cicflowmeter` package turns packets into the 77-feature CICFlowMeter flow schema.
3. **Packet features.** A separate Scapy pass derives packet-level attributes the flow schema omits: TTL mean/variance, TCP window size, retransmission counts, IP-fragment counts, payload-size distribution.
4. **Per-flow state.** A trained Keras ANN (`ISAA_ANN.h5`, MinMax-scaled input) classifies each flow as BENIGN / DDoS / DoS / PortScan.
5. **Temporal dataset.** Flows are sorted chronologically, bucketed into 10-second windows, and aggregated into a 28-feature network-state vector per window, with state transitions and length-5 sliding sequences and a strictly chronological train/val/test split.
6. **Forecast.** An LSTM predicts the next window's state; a separate direct model predicts six windows ahead (H1–H6) with a frozen early-warning threshold. Persistence and multinomial-logistic-regression baselines are scored on identical sequences.
7. **ATT&CK context.** An offline, evidence-aware rules layer maps the predicted state to a small set of ATT&CK techniques (T1595, T1046, T1498, T1499) with a deterministic heuristic confidence, supporting evidence, and explicit limitations. It never asserts attribution or intent.
8. **Explainability.** Gradient $\times$ input attribution over the 77 scaled features identifies which flow statistics drove each prediction, both per flow and aggregated for the capture.

4 Key features

- Two input modes — live capture and offline file analysis — sharing one extraction/prediction path.
- No fabricated values anywhere: every figure shown is read back from a real subprocess or model output, or displayed as N/A.
- Chronological-only splits; a dedicated data-leakage check across windows and sequences.
- One-step and six-step forecasting with honest, labelled proxy-time limitations.
- Deterministic, offline ATT&CK mapping with per-candidate evidence and limitations.
- Per-prediction and per-capture feature attribution.
- A Wireshark-inspired local dashboard (static HTML/CSS/JS, no build step) served by the back-end.

5 Architecture and technology

Backend	FastAPI + Uvicorn, a small explicit state machine
Model	TensorFlow 2.18 / Keras Sequential MLP (4-class softmax)
Scaler	scikit-learn MinMaxScaler (persisted)
Flow features	cicflowmeter 0.5.0 (Scapy-based)
Packet features	Scapy 2.6.1 streaming PcapReader
Forecasting	Keras LSTM (one-step and direct H1–H6)
ATT&CK	Local pinned Enterprise ATT&CK v19.1 subset
Frontend	Plain HTML/CSS/JS, polled REST, no framework
Capture tools	dumpcap/tcpdump, tshark, capinfos

6 Platform support

- **Windows** — install Wireshark; run the PowerShell script `run_backend.ps1`.
- **Linux** — `setcap` on `tcpdump`; run `run_backend.sh`.
- **macOS** — `scripts/macos_setup.sh` installs the Wireshark CLI tools and loads the `ChmodBPF` helper (macOS has no `setcap`); the interface picker shows friendly names (Wi-Fi, Ethernet, Thunderbolt) mapped from `networksetup`.

Open the dashboard at `http://127.0.0.1:8765`.

7 Provenance and attribution

Base repository

Cloned from the original NIDS repository (commits by Manigandan Ramadasan, 453a5a1–39316cb; audited in `PIPELINE.md` as of `main @ 39316cb`). Pre-existing content: `models/ISAA_ANN.h5` (the trained Keras ANN), `models/minmax.bin` (its scikit-learn MinMaxScaler), three Jupyter notebooks (`NIDS -- ANN.ipynb`, `NIDS -- ML.ipynb`, `NIDS (Prediction).ipynb`), `scripts/packetsniff.sh`, and a report PDF. The prediction notebook’s hardcoded `0:\VIT...` and `C:\Users\varun...` paths indicate a Vellore Institute of Technology student project; “ISAA_ANN” is the original team’s model name. Everything from commit 945f9ad onward — the end-to-end

pipeline, FastAPI backend, dashboard, Phase 3/4 LSTM forecasting, MITRE layer, gradient $\times$ input explainability, packet-level features, and cross-platform capture — was added on top.

Feature extraction

`cicflowmeter` 0.5.0 (PyPI, MIT-licensed, Scapy-based) — a drop-in for the Java `ahlashkari/CICFlowMeter`, which was abandoned here because its `jnetpcap` dependency has no working modern 64-bit build path. Two upstream bugs are patched idempotently by `scripts/patch_cicflowmeter.py`.

Training and evaluation dataset

CICIDS2017, produced by the Canadian Institute for Cybersecurity, University of New Brunswick — <https://www.unb.ca/cic/datasets/ids-2017.html>. Eight official weekday session flow CSVs are used: Phase 3 trains on Monday, Tuesday, Wednesday and Thursday-Afternoon-Infiltration; Phase 4 adds Thursday-morning Web Attacks, Friday morning, Friday PortScan and Friday DDoS. The raw CSVs are not vendored in Git — they are published as GitHub Release assets (`phase3-datasets`, `nids-cicids2017-phase4-added-source-csvs.tar.gz`) with per-file SHA-256 hashes in `datasets/SHA256SUMS` and the archive hash in `datasets/README.md`. The ANN was trained by the original team on the classic CICIDS2017 78-column export (77 numeric features after dropping the duplicate header column and the label).

MITRE ATT&CK

Enterprise ATT&CK v19.1 (MITRE Corporation), distributed as STIX 2.1 via <https://github.com/mitre-attack/attack-stix-data>. A pinned, verified subset — techniques T1595, T1046, T1498, T1499; tactics Reconnaissance, Discovery, Impact — is committed at `backend/mitre/data/enterprise_attack_v19.1_subset.json` and loaded with publisher and bundle-SHA-256 provenance checks (`backend/mitre/mapper.py`). Used entirely offline; no network calls.

Demonstration captures

`captures.zip` is the community “DDoS Packet Capture Collection” (mirror <https://wqrlld.net/captures.zip>) — anonymized real-world and lab DDoS captures, free to use for building DDoS-protection systems, with attribution appreciated but not required. Members are named `amp.*` (amplification) or `pkt.*` (spoofed / direct). It is not part of the repository; four small members are copied into `pcaps/` locally for demos.

Core libraries

FastAPI and Uvicorn, TensorFlow 2.18 / Keras, scikit-learn 1.5.2, Scapy 2.6.1, pandas / NumPy, joblib, python-multipart, and pytest — all pinned in `backend/requirements.txt`.

8 Known limitations

- The ANN was trained on CICIDS2017 (2017 lab traffic) and can misclassify modern HTTPS/QUIC/STUN as DoS/DDoS — a model-generalisation limit, not a pipeline defect.
- Only the ANN is deployable; Random Forest / Decision Tree / KNN from the original notebooks were never persisted.

- Forecast horizons use row-order proxy windows on the training CSVs, not validated wall-clock seconds; the UI labels this explicitly.
- The ATT&CK layer covers Reconnaissance / Discovery / Impact only; it does not cover Initial Access, Lateral Movement, C2 or Exfiltration (the classifier has no such states).
- The packet-feature extractor is standalone — this `cicflowmeter` build emits no source/destination IP columns, so the two feature levels are not yet joined on a common flow key.
- The persisted scaler was pickled under an older scikit-learn; values are unaffected and the version warning is suppressed at load.